Predicting the likelihood of a TikTok video going viral using intrinsic and extrinsic signals

Oscar Rodriguez, Mahda Soltani, Ela Naz Sigin

March 10, 2026

Abstract

Social media platforms, like TikTok have become one of the dominant channels through which information, entertainment, and political and cultural trends propagate at scale. A defining feature of these platforms is the extreme skewness in attention: while the vast majority of uploaded videos receive little to no exposure, a small fraction achieve *viral* reach, accumulating orders of magnitude more views within a short time window. This heavy-tailed distribution of engagement naturally raises the question of how to operationalize and predict "virality" in a principled way.

This investigation explores the use of Machine Learning techniques to pin down a formal framework for the task of predicting the likelihood of a TikTok video going viral using both intrinsic (e.g. video content) and extrinsic features (e.g. video author).

Introduction

...

Methods

This section goes over the different aspects of the Machine Learning pipeline that we crafted for the given task, starting from data processing all the way to the model architecture and reasoning behind each of the decisions.

Data collection and processing

We started off with a base dataset called TikTok-10M from which we sourced a rich subset of examples that we complemented with a scraping step to download the videos and publisher accounts to extract additional signals (e.g. video bytes, aspect ratio, etc.) as well as account information (e.g. account statistics, publishing history, etc.).

With all this raw data at hand, we produced a curated dataset for our task. The following tables contain a detailed documentation of dataset dimensions, encompassing intrinsic content signals and extrinsic author/metadata signals we deemed relevant to the problem.

Task definition

The definition of virality (is_viral) we produced is the target dimension we optimize for. It is meant to capture outliers relative to any given account stack-ranked in absolute terms with the goal of producing a signal that's useful for any account size.

In order to recreate a mixed signal that fits the previous definition, we compute two scores for each video, **engagement** and **view velocity**.

Dimension	Type	Description	Extraction / Treatment
User / author signals			
user_id/username	ID/String	Unique identifier and handle of the creator	Scraped (TikTok web profile)
user_language	String	Configuration language of the account	Base dataset
city	String	City where the video was posted from	Base dataset
user_verified	Boolean	Verification status of the author	Scraped (TikTok web profile)
is_private	Boolean	Whether the account is set to private	Scraped (TikTok web profile)
author_stats	Floats	Follower, hearts, video, and friend counts	Scraped; $\log(1+x)$ scaled
account_create_time	Timestamp	When the user account was first created	Scraped (unix epoch)

Video content & metadata			
description	String	User-generated caption and hashtags	Base dataset
create_time	Timestamp	Exact time of video publication	Base dataset (UTC)
temporal_features	Float	Hour of day and Day of week	Sine/cosine cyclic encoding
is_ad	Boolean	Flag for paid/sponsored content	Base dataset
duration	Integer	Total length of the video in seconds	Base dataset
challenges	String	Associated hashtag challenges or trends	Base dataset
video_bytes	Binary	Scraped video resized to 256x256 @ 1fps	`yt-dlp` + `moviepy`
detected_objects	String	Objects detected in video	Pretrained facebook/detr-resnet-50

Audio & music specs			
music_info	String	Title, album, and author name of the audio	`yt-dlp` payload metadata
music_play_url	String	URL source for the audio file	Base dataset

Technical / video specs			
resolution	Integer	Video width and height in pixels	`yt-dlp`
aspect_ratio	Float32	Ratio of width to height	Calculated from payload
filesize	Integer	Total storage size of the video file	`yt-dlp` payload metadata
codecs	String	Video (vcodec) and audio (acodec) formats	`yt-dlp` payload metadata
vq_score	Float64	Visual quality score	Base dataset

Engagement stats & target labels			
engagement_score	Float64	Weighted sum of interactions per view	Engineered; weighted calculation
view_velocity	Float64	Views accumulated per hour since creation	Engineered; $\log(1+x)$ scaled
is_viral	Binary	Classification ($P = 0.95$) of engagement	Engineered; (quantile threshold)

Engagement

$$\text{engagement}_i = \frac{\text{share_count} \times w_0 + \text{save_count} \times w_1 + \text{comment_count} \times w_2 + \text{like_count} \times w_3}{\text{play_count}} \tag{1}$$

Where *engagement* is meant to represent a volume of "interactions" a video had relative to its play count, making the score relative to the account size. As shown in the following graph, its expected to assume that accounts with a larger audience will most likely outperform its counterparts, making these videos seem more "viral" in absolute measurements.

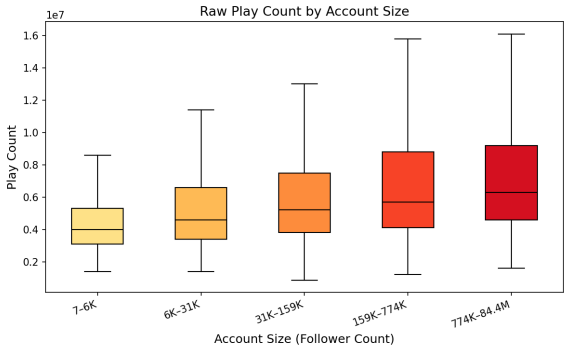

Figure 1: As the account size increases, the play count increases. We're interested in somehow normalizing this to produce a signal that fairly compares "virality" irrespective of account size.

What this signal is meant to capture instead is a degree of "engagement" that's normalized to the audience size of any given account. To convey the idea, a small account with a few hundred followers that produces a video that generates thousands of interactions would have a comparable engagement to an acconut with millions of followers that accumulated hundredths of thousands of interactions instead.

View velocity

$$\text{view velocity}_i = \frac{\text{play_count}}{\text{delta_hours}} \tag{2}$$

View velocity normalizes the value of *play_count* by the total amount of time a video has been live collecting interactions. A video with a high "view velocity" is collecting interactions "faster" relative to the rest.

Producing the signal

We add together both measurements to produce a signal S_i for each video that we use to stack-rank them in absolute S_i terms and consider the top 5% as "viral":

$$S_i = \frac{\text{engagement}_i}{\text{engagement}_{max}} + \frac{\text{view velocity}_i}{\text{view velocity}_{max}} \tag{3}$$

$$y^{(i)}_{\text{is_viral}} = \begin{cases} 1 & \text{if } S_i \geq F_S^{-1}(0.95) \\ 0 & \text{Otherwise} \end{cases} \tag{4}$$

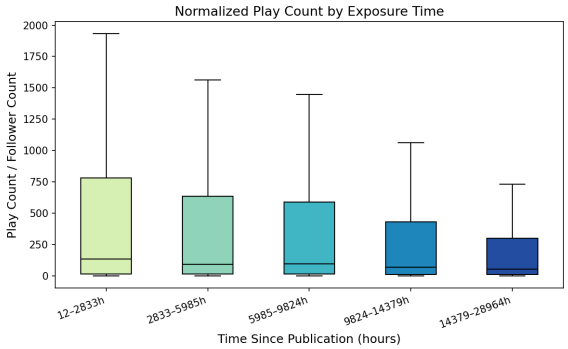

Figure 2: The ratio of plays to delta_hours shows the natural time-decay bias in raw play_count. A video with 100k views in 1 hour would look worse than a video with 100k + 1 views over 1 month. Normalizing by delta_hours "levels the playing field".

The reason we approached the concept of virality as described is rooted in convenience and usefulness. There are just too many dynamic forces that could influence the level of virality a video could reach. Many things like the current social discourse, ongoing political and economic events, or just sheer luck (being in the right place at the right time) could determine the success of any given video.

Trying to capture such a broad range of "reasons" is likely near impossibility limits either due to data or resource constraints.

Instead we try to condense a signal that's useful even for small accounts with the available data. We're thinking about small businesses/ content creators looking to amplify their social presence. Rolling out a video that's a couple of folds more viral relative to their expected baseline is considered a win!

Model architecture

The architecture implements a mixture of state-of-the-art ML processing techniques for the different data modalities fed into the model; all these get concatenated and passed through a late fusion module to finally produce 2 regression heads for the engagement and view velocity scores and 1 classification head for the virality class. We ultimately stitch them together to compute a final weighted loss to have the different output targets contribute in different quantities.

Data modalities

Text input

The text input is a concatenation of the different text dimensions in the following form:

"[CLS] [TITLE] ... [MUSIC] ... [CHALLENGES] ... [OBJECTS] ... [LOC] ..."

That we populate with the example's data and feed this into a pretrained, frozen DistilBERT backbone to extract a 768-dimensional semantic representation from the [CLS] token, capturing the semantics of all textual components from the example in a compact representation.

Video input

During the dataset composition step, the original videos were downsized and downsampled to 224x224 pixels at 1 FPS to reduce processing requirements. From that transformed version, we

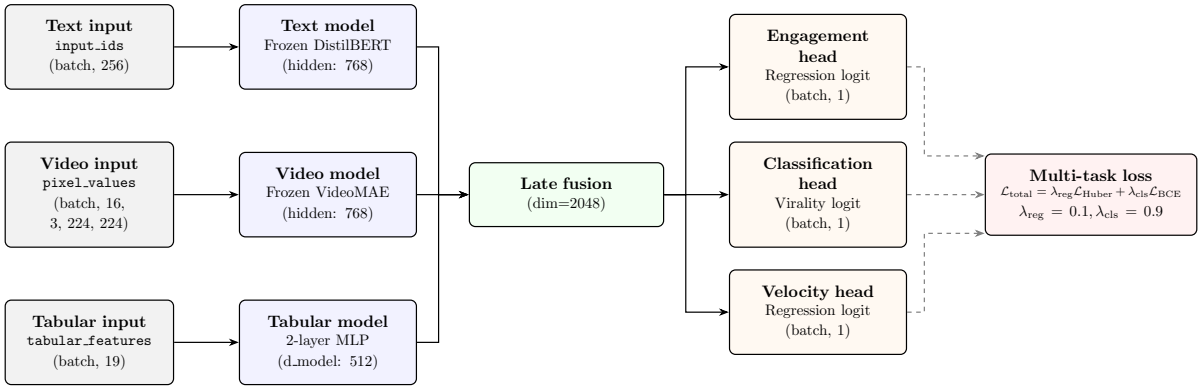

Figure 3: End-to-end architecture: features are extracted from multimodal inputs, fused into a shared latent space, and passed to specialized heads. The final objective is a weighted combination of Huber regression loss and weighted binary cross-entropy (BCE) loss.

sample 16 frames that we feed into a pretrained, frozen VideoMAE model to encode the video into a spatial-temporal embedding space.

We then apply global average pooling across the output tokens to yield a 768-dimensional embedding that captures both visual features and temporal dynamics.

Tabular input

The tabular branch processes 19 heterogeneous features including author signals, temporal metadata, and video quality scores. To handle the varying scales of these inputs, we apply $log(1+x)$ scaling to high-variance author statistics and sine/cosine cyclic encoding to temporal features like hour of day to preserve periodic relationships. These are fed into a 2-layer MLP with layer normalization and dropout, projecting them into a learnable 512-dimensional space.

Late fusion

In the late fusion module, we concatenate the 768-dimension text embedding, the 768-dimension video embedding, and the 512-dimension tabular output into a single 2048-dimensional vector. This combined representation is passed through a shared fusion MLP, which maps the multimodal features into a unified latent space of $d_model = 512$. This shared representation allows the model to learn complex inter-modal correlations before branching into specialized tasks.

Multi-head targets

We employ a multi-head architecture to simultaneously model continuous and categorical virality signals. Each head progressively halves the upstream d_model to reduce dimensionality while increasing specialization:

- Engagement and velocity heads: Two regression heads predict the continuous engagement and view velocity scores.
- Classification head: A single head predicts the probability of virality, defined by the 95th percentile of the combined engagement and velocity distribution.

Loss target

We experimented with two types loss targets.

Weighted Binary Cross-Entropy loss (BCE)

This loss target is a composite of the different upstream heads, weighted to prioritize the primary classification task while using the regression heads as auxiliary signals:

- We use Huber loss for the regression targets to remain robust against the extreme outliers common in viral distributions, and a weighted BCE for the classification target.
- We set $\lambda_{cls} = 0.7$ and $\lambda_{reg} = 0.3$. This heavy weighting on classification ensures the model focuses on detecting the rare viral events (which are themselves upweighted by a factor of $virality_threshold/(1 - virality_threshold)$ in the BCE loss), while the regression tasks provide continuous "engagement" and "view velocity" measurements to complement the final signal.

Focal loss

This loss target – introduced in "Focal Loss for Dense Object Detection" – replaces the standard weighted BCE with an adaptive weighting mechanism designed to prioritize high-error samples over well-classified "easy" examples (i.e. the over-populated negative class). Focal loss can be. modulated with a "focus parameter" γ and a "balance factor" α.

- We set $\gamma = 2.0$ to aggressively down-weight the loss contributed by the abundant negatives, to allow the model to focus on the hard negatives.
- And $\alpha = p_virality_threshold$ to account for the extreme class imbalance to have the model re-weight both classes.

Experiments and results

To evaluate the performance of our multi-modal approach, we track a suite of metrics that address both the regression of engagement signals and the classification of rare viral events. Given the high degree of class imbalance in our dataset and the nature of the problem we're solving for, we prefer to prioritize **recall**, and recall-associated metrics.

- **Receiver Operating Characteristic - Area Under the Curve (ROC-AUC):** Used to measure the general separability of the classes across all potential thresholds.
- **Precision-Recall Area Under the Curve (PR-AUC):** Our primary metric for the classification head. Unlike ROC-AUC, PR-AUC does not reward the model for correctly predicting the "non-viral" majority, providing a more honest view of the model's ability to identify viral content.
- **F1-Score:** Calculated at the 0.5 threshold to measure the harmonic mean of precision and recall.

Training Dynamics

We compared two distinct training regimes: a multi-task **weighted loss** baseline and a specialized **focal loss** run. The baseline utilized a linear learning rate decay over 15 epochs, while the focal loss experiment used a cosine scheduler with a 10% warmup phase to stabilize the initial gradients given the aggressive $\gamma = 2.0$ parameter.

As shown in figure 1 and figure 2, both models exhibited steady convergence. The focal loss model reached its optimal performance earlier, around epoch 8, suggesting that this loss type allowed the model to ignore easy negatives and converge faster on the decision boundary for viral samples.

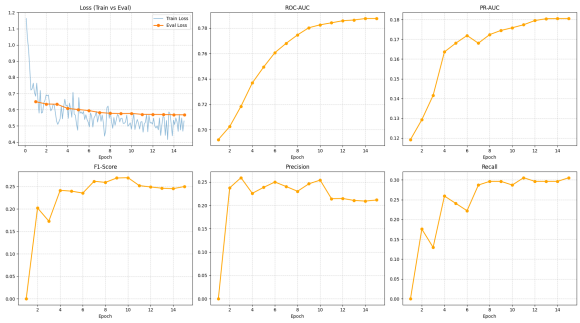

Figure 4: Weighted loss training dynamics.

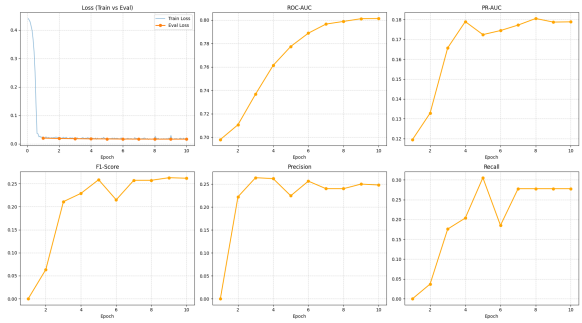

Figure 5: Focal loss training dynamics.

Analysis of results

Metric	Weighted loss	Focal loss	Δ
ROC-AUC	0.7875	**0.7987**	+1.4%
PR-AUC	0.1805	**0.1805**	—
F1-Score	0.2452	**0.2575**	+5.0%
Precision	0.2092	**0.2400**	+14.7%
Recall	**0.2963**	0.2777	-6.2%

Table 1: Comparative performance between weighted BCE and focal loss targets.

The transition to focal loss yielded a significant **14.7% improvement in precision**. While the baseline weighted BCE loss model achieved higher recall by aggressively upweighting the minority class, it suffered from a high false-positive rate. Focal loss mitigated this by forcing the model to focus on "hard" examples, resulting in a more balanced F1-score of 0.2575.

Despite the marginal improvement in ROC-AUC, the stagnancy of the PR-AUC at 0.1805 across both experiments confirms that while we can tune the precision-recall trade-off, the absolute confidence in viral classification is likely bounded by factors we didn't model:

1. **Unmeasurable stochasticity:** Like we suggested earlier, there are simply too many ongoing dynamics that influence the popularity of a video (e.g. socio-political discourse, randomness, etc.) that we couldn't realistically capture in a finite dataset.

2. **Gaps in dataset:** We might still be missing additional input sources like audio, video dynamics, or platform-specific nuances (e.g. current trends, etc.) that significantly influence a video's success.

Next steps

While the current model establishes a strong baseline for late-fusion multimodal learning, the discrepancy between ROC-AUC and PR-AUC suggests that the model is struggling with the extreme sparsity of viral signals. We propose the following directions for future research:

- Run **ablation experiments** on the input features to surface relevance (e.g. feeding everything but the video input, and viceversa).

- An interesting follow-up we pondered on involves incorporating an auxiliary regression task to predict the continuous growth size of engagement overtime using a **Self-Exciting Hawkes Process** regression module.

Our investigation demonstrates that while intrinsic content features provide a signal for virality, the task remains highly stochastic. By leveraging a multi-task learning framework with weighted losses, we effectively stabilized the training of a high-dimensional multimodal model, achieving a respectable ROC-AUC that serves as a foundation for future algorithmic discovery in social media dynamics.

References

...